

CIS241

System-Level Programming and Utilities

C - File I/O

Erik Fredericks, frederer@gvsu.edu

Fall 2025

Based on material provided by Erin Carrier, Austin Ferguson, and Katherine Bowers

File Input/Output

Files are treated like streams

Files are managed via file pointers

All file functions are in `<stdio.h>`

Generally, the process looks like this:

- Create file pointer
- Open file in the correct mode
- Read and/or edit the file
- Close the file

Opening a file

```
FILE* fp_in;  
fp_in = fopen("in_filename", "r");  
// Read file  
fclose(fp_in);  
  
FILE* fp_out;  
fp_out = fopen("out_filename", "w");  
// Write to file  
fclose(fp_out);
```

fopen

```
FILE* fp = fopen(filename, mode);
```

Modes:

- "r" to read
- "w" to write (discards old contents)
- "a" to append
- Can also add "+" to both read/write (docs)

You can add "b" to the end for binary mode

- This does not matter on Unix systems

fopen

Mode	File Exists?	File Doesn't Exist?
read	Open to read	Error
write	Overwrite it	Create it
append	Append to it	Create it

fopen returns **NULL** on failure - good to check!

When can it fail?

- Reading a file that doesn't exist
- Incorrect permissions

```
FILE* fp = fopen("file.txt", "r");  
if(fp == NULL) {  
    printf("Error! Could not open file\n");  
    return 1;  
}
```

`fclose`

```
fclose(file_pointer);
```

Closing your files is important!

- File output is often buffered – changes are only
- written to file when its closed!

Even if your system doesn't buffer, we want to close files to make our code portable!

Reading from a file

Very similar to reading input from stdin!

- `scanf` switches to `fscanf`
- `fgets` and `getline` work out of the box!

```
fscanf(fileptr, formatstr, memaddr1, ...);
```

```
fgets(char* s, int size, FILE* stream);
```

```
getline(char ** lineptr, size_t n, FILE* stream);
```


Writing to a file

```
fprintf(FILE* fp, format_str, args...);
```

- Identical to `printf`, but writes to a file, not stdout
- Don't forget the first `f`!

```
fputc(int c, FILE* fp);
```

- Writes a single character to file
- there's also a `fgetc`

```
fputs(char* s, FILE* fp);
```

-Writes the string to file

Working with bytes

```
fread(void* buffer, size_t size, size_t n, FILE* fp);
```

- Reads n objects (which are size bytes each) from file and stores data in buffer.

```
fwrite(void* buffer, size_t size, size_t n, FILE* fp);
```

- Writes n objects (which are size bytes each) from buffer to fp

Working with bytes

```
fseek(FILE* fp, long offset, int origin);
```

- Think of your cursor in a text file, next read/write will occur here
- This also applies to reading/writing files in C
- `fseek` will move this cursor (position indicator)

Options for origin:

- `SEEK_SET` - start of file
- `SEEK_CUR` - current position
- `SEEK_END` - end of file

fseek : https://www.tutorialspoint.com/c_standard_library/c_function_fseek.htm

```
int main() {
    FILE *file = fopen("example1.txt", "r");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }

    fseek(file, 0, SEEK_SET);

    char ch = fgetc(file);
    if (ch != EOF) {
        printf("First character in the file: %c\n", ch);
    }

    fclose(file);
    return 0;
}
```

Final thoughts

Pay attention to return values!!

- e.g., input functions will treat errors and `EOF` differently
- This will save you future headaches

If you are switch between reading and writing in the same file pointer

- Use `fseek()` or `fflush()` before switching
- This forces buffer to be flushed

Examples!

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE* fp;

    fp = fopen("demo.txt", "w+"); // + makes it read/write
    if (fp == NULL) {
        printf("Error opening file for writing\n");
        return -1;
    }

    fprintf(fp, "%s\n", "CIS241");
    fclose(fp);

    return 0;
}
```

Examples

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE* fp;
    char buffer[255];

    fp = fopen("demo.txt", "r");
    if (fp == NULL) {
        printf("Error opening file for reading\n");
        return -1;
    }

    fgets(buffer, 255, fp);
    printf("Data read: %s\n", buffer);
    fclose(fp);
    return 0;
}
```

Examples

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE* fp;
    char buffer[255];

    fp = fopen("demo.txt", "r");
    if (fp == NULL) {
        printf("Error opening file for reading\n");
        return -1;
    }

    while (fgets(buffer, sizeof(buffer), fp)) {
        printf("%s", buffer);
    }
    fclose(fp);
    return 0;
}
```


read and write

<https://stackoverflow.com/questions/29677577/writing-and-reading-from-a-file-at-the-same-time>

```
int main(void) {
    FILE* filePtr = fopen("tabs.txt", "r+");
    int c;

    while((c = fgetc(filePtr)) != EOF) {
        if(c == '\t') {
            fseek(filePtr, -1, SEEK_CUR);
            fputc(' ', filePtr);
        }
    }
    fclose(filePtr);
    return 0;
}
```

https://medium.com/@future_fanatic/a-beginners-guide-to-file-handling-in-c-with-fopen-e0e7c6969b92

<https://stackoverflow.com/questions/3501338/c-read-file-line-by-line>

<https://www.geeksforgeeks.org/c/read-a-file-line-by-line-in-c/#>

Some practice

Grab this file (hmm, wasn't there a terminal command you could easily use?)

- <https://efredericks.github.io/gvsu-cis241/assets/hw-data/thggttg.txt>

Make a C program that:

1. Opens it for reading
2. Opens a second file for writing
3. Writes to the new file (without a newline) "The answer is: "
4. Goes to the 1442nd byte in the downloaded file and writes out 8 characters

Submit with your final mini-project for a bit of extra credit.